

Residual Connections & Gradient Flow.

A transformer block does not replace its input — it *adds* to it: $x' = x + F(\text{LN}(x))$. That single “ $x+$ ” is what lets us stack 96 layers without the gradient dying on the way back. Below we derive the Jacobian, multiply out a 10-layer toy by hand, and watch the signal survive — every number computed in full, nothing deferred.

THE EQUATION

$$x' = x + F(\text{LN}(x)) \quad \implies \quad \frac{\partial x'}{\partial x} = I + J_F$$

Worked example. A 2-D surrogate sublayer with $J_F = 0.5 I$ stacked $L = 10$ deep: the residual path preserves a gain of $1.5^{10} \approx 57.67$ where a plain net collapses to $0.5^{10} \approx 9.77 \times 10^{-4}$.

Topic 1 of 5 in this module · companion to the course page · v1.0

Contents

1	The claim	2
2	Block diagram	3
3	Deriving the Jacobian $I + J_F$	3
4	Worked computation: 10 layers, by hand	4
5	Consequences that matter	5
6	Where this sits in the track	5
A	Reproducible (NumPy)	5

1 The claim

A transformer block transforms a token representation $x \in \mathbb{R}^d$ not by overwriting it but by computing a *correction* and adding it back:

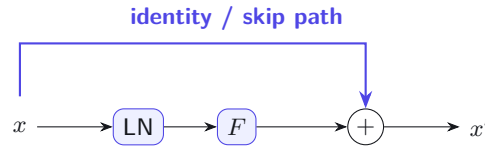
$$x' = x + F(\text{LN}(x)) \quad (1)$$

Here F is a sublayer (attention or feed-forward) and LN is layer normalization. The term $F(\text{LN}(x))$ is the *residual*; the bare x that survives is the *skip* (or *identity*) path. The claim of this topic: that identity path is a gradient *highway* which guarantees backpropagated gradients cannot vanish, no matter how deep the stack.

Symbol	Shape	Meaning
x, x'	$\mathbb{R}^d$	input / output token representation of one block
$F(\cdot)$	$\mathbb{R}^d \rightarrow \mathbb{R}^d$	sublayer map (attention or FFN)
LN	$\mathbb{R}^d \rightarrow \mathbb{R}^d$	layer normalization (Topic 2)
J_F	$\mathbb{R}^{d \times d}$	Jacobian $\partial F(\text{LN}(x))/\partial x$
I	$\mathbb{R}^{d \times d}$	identity matrix from the skip path
L	scalar	number of stacked blocks

Think of each block as a sticky note pasted on top of the original message rather than a rewrite of it. The original message is always still there underneath. When the error signal walks backward through the stack, at every block it finds a clean “pass-through” door (the $+x$) right next to the sublayer door — so even if every sublayer door is half-closed, the pass-through door stays wide open and the signal gets home.

2 Block diagram



The thick indigo arrow is the skip path. Its derivative is exactly I — it neither shrinks nor grows the gradient. That is the whole trick.

3 Deriving the Jacobian $I + J_F$

Differentiate (1) with respect to x . The sum rule gives

$$\frac{\partial x'}{\partial x} = \underbrace{\frac{\partial x}{\partial x}}_{=I} + \frac{\partial}{\partial x} F(\text{LN}(x)).$$

The second term is a chain rule through F then through LN:

$$\frac{\partial}{\partial x} F(\text{LN}(x)) = \underbrace{\frac{\partial F}{\partial (\text{LN}(x))}}_{J_F^{(\text{sub})}} \cdot \underbrace{\frac{\partial \text{LN}(x)}{\partial x}}_{J_{\text{LN}}} =: J_F.$$

Hence

$$\boxed{\frac{\partial x'}{\partial x} = I + J_F, \quad J_F = J_F^{(\text{sub})} J_{\text{LN}}} \quad (2)$$

Now stack L blocks $x_0 \rightarrow x_1 \rightarrow \dots \rightarrow x_L$. By the chain rule the gradient of a downstream loss $\mathcal{L}$ w.r.t. the input is the product of per-block Jacobians:

$$\boxed{\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^L (I + J_l)} \quad (3)$$

Compare with a *plain* deep net $x_l = F_l(x_{l-1})$, which has *no* identity term:

$$\frac{\partial \mathcal{L}}{\partial x_0} = \frac{\partial \mathcal{L}}{\partial x_L} \prod_{l=1}^L J_l. \quad (4)$$

In (4) every factor is a sublayer Jacobian. If those tend to have singular values < 1 (the usual case after init), the product shrinks geometrically and the gradient *vanishes*. In (3) each factor is $I + J_l$: expanding the product, one term is $I \cdot I \cdots I = I$, an undiminished path straight from the loss to x_0 .

Expanding $\prod_l (I + J_l) = I + \sum_l J_l + \sum_{l < m} J_m J_l + \dots$. The leading I is a gradient that reaches x_0 *without being multiplied by a single sublayer Jacobian*. So the worst case for the residual stack is “gradient = identity,” never “gradient = zero.” Vanishing is structurally impossible.

4 Worked computation: 10 layers, by hand

Take a 2-D surrogate where every sublayer has the same Jacobian $J_l = 0.5 I$ (an isotropic gain of one half). This is the regime that kills plain deep nets. With $I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$:

$$\text{Plain net factor: } \prod_{l=1}^{10} (0.5 I) = 0.5^{10} I, \quad \text{Residual factor: } \prod_{l=1}^{10} (1.5 I) = 1.5^{10} I.$$

Because both are scalar multiples of I , we track the scalar. Multiplying out layer by layer:

layer l	plain 0.5^l	residual 1.5^l
1	5.000×10^{-1}	1.5000
2	2.500×10^{-1}	2.2500
3	1.250×10^{-1}	3.3750
4	6.250×10^{-2}	5.0625
5	3.125×10^{-2}	7.5938
6	1.563×10^{-2}	11.3906
7	7.813×10^{-3}	17.0859
8	3.906×10^{-3}	25.6289
9	1.953×10^{-3}	38.4434
10	9.766×10^{-4}	57.6650

After 10 layers the plain net has scaled the gradient by 9.766×10^{-4} — a thousand-fold attenuation, well on the road to numerical death. The residual stack has scaled it by 57.665. The ratio between the two paths is

$$\frac{1.5^{10}}{0.5^{10}} = 3^{10} = 59049,$$

nearly five orders of magnitude. The heatmap below shows the two cumulative gains; intensity scales with log-magnitude so both extremes stay legible.

	1	2	3	4	5	6	7	8	9	10
plain	.50	.25	.13	.06	.03	.016	.008	.004	.002	.001
residual	1.5	2.3	3.4	5.1	7.6	11	17	26	38	58

5 Consequences that matter

1. **No vanishing by construction.** The I in $I + J_l$ guarantees a lower bound on the gradient magnitude; deep stacks (12, 24, 48, 96 layers) remain trainable.
2. **Exploding is still possible.** The same product can *grow* (here 57.665). Residuals prevent vanishing, not explosion — normalization (Topic 2) and pre-norm placement (Topic 4) keep the upper end in check.
3. **Initialization at identity.** If F starts near zero (small final-layer init), the block starts as $x' \approx x$, so an untrained deep stack is close to the identity function — a benign, easy-to-optimize starting point.
4. **Depth becomes cheap.** Because each added block contributes J_l on *top of* a preserved identity, adding depth refines rather than risks the representation.

The residual connection adds x to $F(\text{LN}(x))$, *not* to $\text{LN}(x)$. The skip path carries the *raw, un-normalized* x forward. A common bug is to write $x' = \text{LN}(x) + F(\text{LN}(x))$ — this destroys the clean identity Jacobian (the skip now passes through LN's Jacobian too) and reintroduces the vanishing problem. Keep the skip raw.

6 Where this sits in the track

Link	Connection
Module 0.1	Attention is one choice of sublayer F .
Topic 2 (this module)	LN, the map whose Jacobian J_{LN} feeds J_F .
Topic 4 (this module)	Pre- vs post-norm changes <i>where</i> LN sits relative to the skip, altering (2).
Phase 1	Why GPT-style models stack 12–96 of these blocks at all.

A Reproducible (NumPy)

Inputs: per-layer sublayer gain 0.5 (so plain factor 0.5^l), residual factor $(1 + 0.5)^l = 1.5^l$, depth $L = 10$. Final values: plain = 9.765625×10^{-4} , residual = 57.6650390625, ratio = $3^{10} = 59049$.

```
import numpy as np
g, L = 0.5, 10
plain    = np.cumprod(np.full(L, g))          # 0.5, 0.25, ..., 9.7656e-4
residual = np.cumprod(np.full(L, 1 + g))      # 1.5, 2.25, ..., 57.6650
print(plain[-1])                             # 0.0009765625
print(residual[-1])                          # 57.6650390625
print(residual[-1]/plain[-1])                # 59049.0  (== 3**10)

# matrix form: J = 0.5*I in 2-D; both products are scalar multiples of I
I = np.eye(2)
P = np.linalg.matrix_power(0.5*I, L)        # = 0.5**10 * I
```

```
Rr = np.linalg.matrix_power(1.5*I, L)    # = 1.5**10 * I
print(P[0,0], Rr[0,0])                  # 0.0009765625 57.6650390625
```

Marevlo Research · Transformer Track · Module 0.2 · Topic 1 of 5.